

Project Documentation

Title-CNN Based Vegetable Image Classification

1. Project Objective

The objective of this project is to build a deep learning model capable of classifying vegetable images into predefined categories using Convolutional Neural Networks (CNN).

The trained model is then deployed using a Flask web application, enabling users to upload vegetable images and receive real-time predictions.

2. Problem Statement

Manual identification of vegetable types from images can be:

- Time-consuming
- Prone to human error
- Inconsistent across users

This project aims to automate vegetable classification using computer vision and deep learning.

3. Project Overview

This project focuses on building a **Convolutional Neural Network (CNN)** model to classify vegetable images into predefined categories. The trained model is integrated with a **Flask web application**, allowing users to upload an image and receive real-time classification results.

4. Project Flow

User → Upload Image (UI)
↓
Flask Backend
↓
CNN Model Prediction
↓
Prediction Displayed on UI

Step-by-step Flow:

1. The user interacts with the User Interface (UI) and selects an image.
2. The selected image is sent to the Flask backend.
3. The CNN model processes the image.
4. The predicted vegetable class is returned to the Flask UI.
5. The prediction result is displayed to the user.

4. Project Screen Shots & photos

5. Data Collection

The dataset consists of labeled vegetable images organized into folders.

Folder Structure:

```
Vegetable Images/  
├── train/  
├── validation/  
└── test/
```

Each folder contains subfolders representing vegetable classes.

Example:

```
train/  
├── Bean/  
├── Carrot/  
├── Radish/  
└── ...
```

6.Data Preprocessing

Data preprocessing ensures the images are suitable for training.

To prepare the images for CNN training, the following steps were performed:

6.1 Image Resizing

All images were resized to:

$150 \times 150 \times 3$

This ensures consistent input size for the CNN.

6.2 Normalization

Pixel values were scaled from:

$0-255 \rightarrow 0-1$

Using:

`rescale=1./255`

Normalization helps:

- Faster convergence
- Stable gradient updates
- Improved training performance

6.3 Data Augmentation

To prevent overfitting, augmentation techniques were applied:

- Rotation
- Width shift
- Height shift
- Zoom
- Horizontal flip

This artificially increases dataset diversity.

6.4 ImageDataGenerator

The ImageDataGenerator class was used to:

- Load images from directories
- Apply augmentation
- Generate batches of images
- Automatically assign labels

Steps Performed:

Create Train and Test Paths

```
train_path = "Vegetable Images/train"
validation_path = "Vegetable Images/validation"
test_path = "Vegetable Images/test"
```

Import Required Libraries

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

Configure ImageDataGenerator

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

test_datagen = ImageDataGenerator(rescale=1./255)
```

Why?

- Rescale pixel values (0–255 → 0–1)
- Augment data to prevent overfitting
- Improve model generalization

Apply ImageDataGenerator to Dataset

```
train_generator = train_datagen.flow_from_directory(
```

```
train_path,  
target_size=(150,150),  
batch_size=32,  
class_mode='categorical'  
)
```

7. Model Building

Custom CNN Model

Import Required Libraries

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
```

Define Input Shape

```
input_shape = (150, 150, 3)
```

Add Convolutional Layers

Convolution layers extract features like edges, textures, and patterns.

```
model.add(Conv2D(32, (3,3), activation='relu'))  
model.add(MaxPooling2D(2))
```

Add Pooling Layers

Pooling reduces spatial dimensions and computation.

Add Fully Connected Layers

```
model.add(Flatten())  
model.add(Dense(128, activation='relu'))  
model.add(Dropout(0.25))  
model.add(Dense(15, activation='softmax'))
```

Softmax activation is used for multi-class classification.

8. Model Compilation

```
model.compile(  
    optimizer='adam',  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

Components:

- Optimizer → Updates weights
- Loss Function → Measures error

- Metrics → Evaluates performance

9. Model Training

```
history = model.fit(  
    train_generator,  
    validation_data=validation_generator,  
    epochs=30  
)
```

During Training:

- Forward propagation
- Backpropagation
- Weight updates
- Validation accuracy monitoring

9. Model Evaluation

```
test_loss, test_accuracy = model.evaluate(test_generator)
```

Evaluation Metrics:

- Accuracy
- Loss
- Validation Accuracy

This determines how well the model generalizes to unseen data.

Save the Model

```
model.save("best_model.keras")
```

This allows reuse without retraining.

10. Test the Model

```
prediction = model.predict(image_input)
```

The predicted class index is converted to vegetable name.

11. Model Deployment

The trained model is integrated into a Flask application.

Application Building

Step 1: Create HTML Files

- index.html → Home page

- prediction.html → Upload and prediction page
- logout.html → Logout page

Step 2: Build Python Flask Code

Flask loads the trained model and handles:

- Image upload
- Image preprocessing
- Model prediction
- Displaying results

Example:

```
img = tf.keras.utils.load_img(filepath, target_size=(150,150))
img_array = tf.keras.utils.img_to_array(img)/255.0
img_array = np.expand_dims(img_array, axis=0)
prediction = model.predict(img_array)
```

12. Final Output

When a user uploads an image:

Image is resized
Pixel values normalized
Model predicts class
Result displayed on UI

Example:

Vegetable is Radish

13. Challenges Faced

- Overfitting during training
- Managing image size consistency
- Flask integration issues
- Environment dependency conflicts

14. Technologies Used

- Python
- TensorFlow / Keras
- CNN
- Flask
- HTML
- CSS

15. Conclusion

This project demonstrates a complete machine learning pipeline:

- Data collection
- Data preprocessing
- CNN model training
- Model evaluation
- Model deployment using Flask
- Real-time prediction through web interface

The system successfully classifies vegetable images with high accuracy and provides a user-friendly web interface for interaction.